Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия, Системное и прикладное программное обеспечение

Дисциплина: Системы искусственного интеллекта

Лабораторная работа №1

Linear Regression & Logistic Regression

Выполнил:
Карнажицкий Максим Романович
Группа: P3311

Проверил:
Королева Ю. А.

г. Санкт-Петербург, 2026

Содержание

1. Ссылка на репозиторий с ноутбуками 2
2. Структура файлов 2
3. Разделение данных 3
4. A. Регуляризация 4
   4.1. Цель работы 4
   4.2. Подбор параметров 4
   4.3. Линейная регрессия 5
   4.4. Логистическая регрессия 6
   4.5. Вывод по части A 7
5. B. KNN для классификации 8
   5.1. Датасет 8
   5.2. Реализация KNN 8
   5.3. Подбор параметров 9
   5.4. Результаты на test 10
   5.5. Вывод по части B 11
6. C. KNN для регрессии 12
   6.1. Датасет 12
   6.2. Принцип работы 12
   6.3. Подбор параметров 12
   6.4. Итоговые метрики 13
   6.5. Сравнение с baseline 14
   6.6. Вывод по части C 14
7. Общий вывод 15

1. Ссылка на репозиторий с ноутбуками

Решение лабораторной работы находится в репозитории:

https://github.com/xtern0o/sii_labs/tree/main/lab2

Основные пояснения, код, графики и результаты находятся в ноутбуках.

2. Структура файлов

```
lab2/
|-- data/
|   |-- day.csv
|   |-- train.csv
|   |-- Dry_Bean_Dataset.xlsx
|   `-- SeoulBikeData.csv
|-- docs/
|   `-- report.pdf
|-- lab2_a.ipynb
|-- lab2_b.ipynb
|-- lab2_c.ipynb
|-- lin_reg.py
|-- log_reg.py
`-- knn.py
```

- `lab2_a.ipynb` - исследование регуляризации
- `lab2_b.ipynb` - классификация методом KNN
- `lab2_c.ipynb` - регрессия методом KNN
- `lin_reg.py` - лин. рег. с L1/L2/ElasticNet регуляризациями
- `log_reg.py` - лог. рег. с L1/L2/ElasticNet регуляризациями
- `knn.py` - собственная реализация KNN

3. Разделение данных

Во всех частях данные перемешивались вручную с фиксированным значением `SEED = 67`.

Выборка делилась на три части:

- `train` - обучение модели;
- `validation` - выбор гиперпараметров;
- `test` - итоговая оценка.

Использовалось соотношение `60 / 20 / 20`.

Параметры обработки данных, например среднее и стандартное отклонение, вычислялись только по `train`. Это предотвращает утечку информации из `validation` и `test`.

4. A. Регуляризация

4.1. Цель работы

В этой части исследовалось влияние регуляризации на линейную и логистическую регрессию.

Использовались четыре варианта:

- без регуляризации;
- L1;
- L2;
- ElasticNet.

Общая функция потерь имеет вид:

$$L = L_{\text{data}} + L_{\text{reg}}$$

Для L1-регуляризации:

$$L_{\text{reg}} = \lambda \sum_{j=1}^{m} |w_j|$$

Для L2-регуляризации:

$$L_{\text{reg}} = \lambda \sum_{j=1}^{m} w_j^2$$

Для ElasticNet:

$$L_{\text{reg}} = \lambda \left(\alpha \sum_{j=1}^{m} |w_j| + (1 - \alpha) \sum_{j=1}^{m} w_j^2 \right)$$

Здесь:

- λ определяет силу регуляризации;
- α задает соотношение между L1 и L2;
- веса смещения `bias` не регуляризуются.

4.2. Подбор параметров

Значения λ и `l1_ratio` выбирались по результатам на `validation`.

Для ElasticNet проверялись:

```
l1_ratios = [0.25, 0.5, 0.75]
```

Test-выборка при подборе параметров не использовалась.

4.3. Линейная регрессия

Для линейной регрессии использовался датасет `day.csv`.

Целевая переменная:

```
y = df["cnt"]
```

Признаки `casual` и `registered` были удалены, так как:

$$\text{cnt} = \text{casual} + \text{registered}$$

Их использование привело бы к утечке целевой переменной.

Лучший результат на validation показала L2-регуляризация при:

$$\lambda = 0.001$$

На test были получены следующие значения:

- без регуляризации: RMSE $\approx$ 766.6;
- L2-регуляризация: RMSE $\approx$ 765.4.

Разница небольшая. Это означает, что исходная модель уже не была сильно переобучена, но L2 немного улучшила результат.

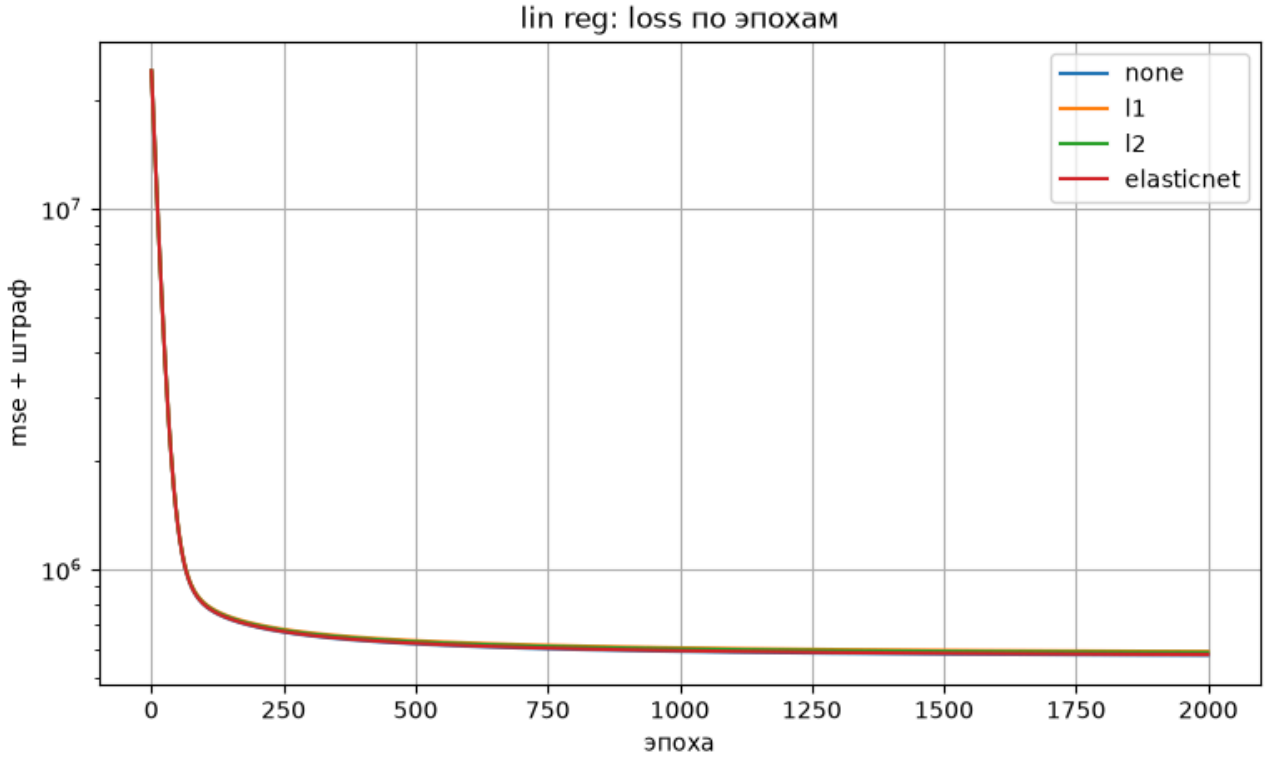


Рис. 1. Результаты линейной регрессии с регуляризацией

4.4. Логистическая регрессия

Для логистической регрессии использовался датасет `train.csv`.

Целевая переменная:

```
y = df["Transported"]
```

Пропуски в числовых признаках заполнялись медианой, а в категориальных - модой. Категориальные признаки кодировались через `get_dummies`.

Лучший ElasticNet использовал:

$$\lambda = 10^{-5}$$

$$\alpha = 0.75$$

На test различные варианты регуляризации показали близкие результаты:

- Accuracy ≈ 0.771;
- $F_1 \approx 0.776$;
- LogLoss ≈ 0.461.

Сильного улучшения от регуляризации не произошло. При выбранных значениях λ она почти не ухудшает качество и немного ограничивает рост весов.

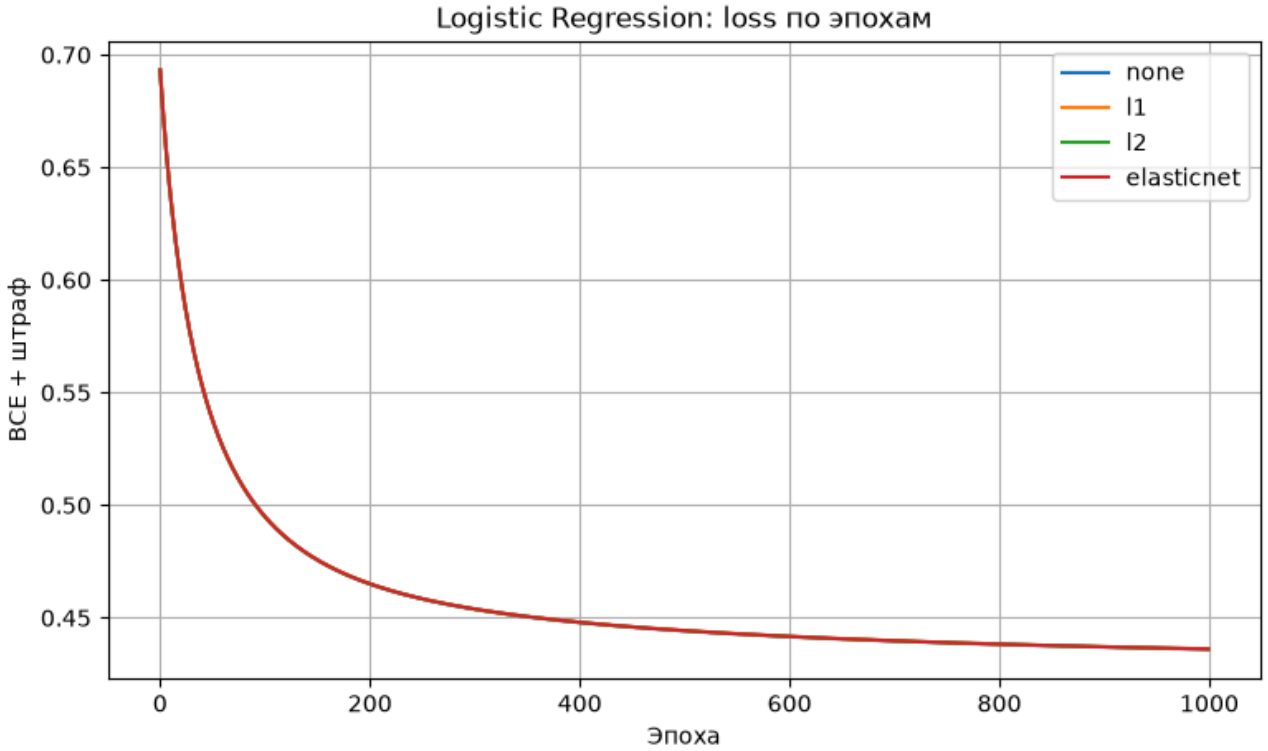


Рис. 2. Результаты логистической регрессии с регуляризацией

4.5. Вывод по части A

Регуляризация помогает ограничивать веса модели и снижать риск переобучения. Однако она не обязана заметно улучшать метрики.

В этой работе L2 немного улучшила линейную регрессию, а для логистической регрессии результаты разных вариантов оказались практически одинаковыми.

5. B. KNN для классификации

5.1. Датасет

Для классификации использовался `Dry Bean Dataset`.

Целевая переменная:

```
y = df["Class"]
```

Остальные 16 числовых столбцов использовались как признаки.

В датасете:

- 13611 объектов;
- 7 классов фасоли;
- пропусков нет.

Были подготовлены два варианта данных:

- без масштабирования;
- со стандартизацией.

Стандартизация выполнялась по формуле:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Среднее и стандартное отклонение вычислялись только по `train`.

5.2. Реализация KNN

Класс `KNN` находится в отдельном файле `knn.py`.

Метод `fit()` сохраняет обучающие объекты и их классы. Обучения весов в KNN нет.

Во время `predict()` для каждого нового объекта:

- вычисляются расстояния до объектов `train`;
- выбираются k ближайших соседей;
- выполняется голосование по классам;
- возвращается самый частый класс.

Евклидово расстояние:

$$d(x, z) = \sqrt{\sum_{j=1}^{m} (x_j - z_j)^2}$$

Манхэттенское расстояние:

$$d(x, z) = \sum_{j=1}^{m} |x_j - z_j|$$

5.3. Подбор параметров

Проверялись следующие значения:

```
k_values = [1, 3, 5, 6, 7, 8, 9, 15]
```

На validation сравнивались четыре варианта:

- Euclidean без scaling;
- Manhattan без scaling;
- Euclidean со scaling;
- Manhattan со scaling.

Основной метрикой выбора был `Macro F1`, так как он отдельно считает качество для каждого класса, а затем усредняет результаты.

Лучший вариант:

- $k = 8$;
- Euclidean distance;
- данные со scaling;
- validation Accuracy = 0.9258;
- validation Macro F1 = 0.9361.

TODO: Вставить график зависимости Macro F1 от k.

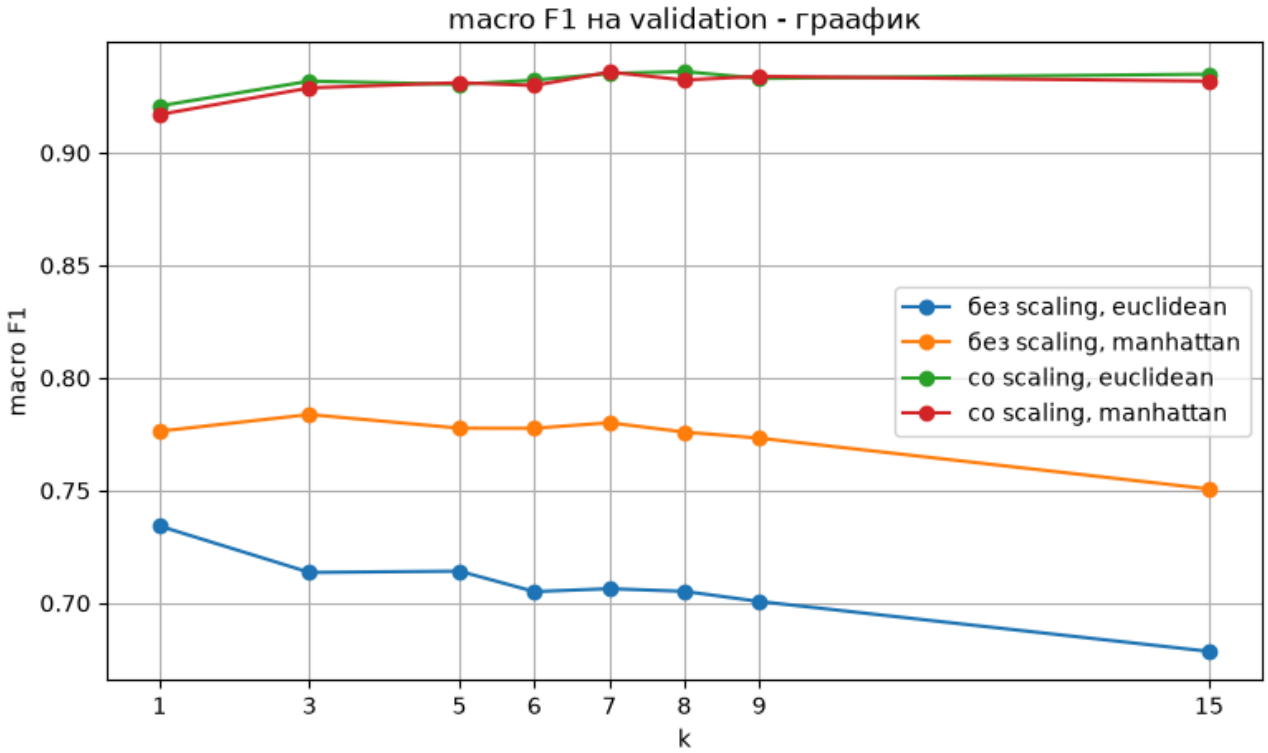


Рис. 3. Сравнение вариантов KNN на validation

5.4. Результаты на test

После выбора параметров модель была обучена на `X_train_scaled` и проверена на `X_test_scaled`.

Получены следующие результаты:

$$\text{Accuracy} = 0.9251$$

$$\text{Macro F1} = 0.9359$$

Модель правильно классифицирует большую часть объектов. Высокий Macro F1 показывает, что качество сохраняется не только для крупных, но и для менее представленных классов.

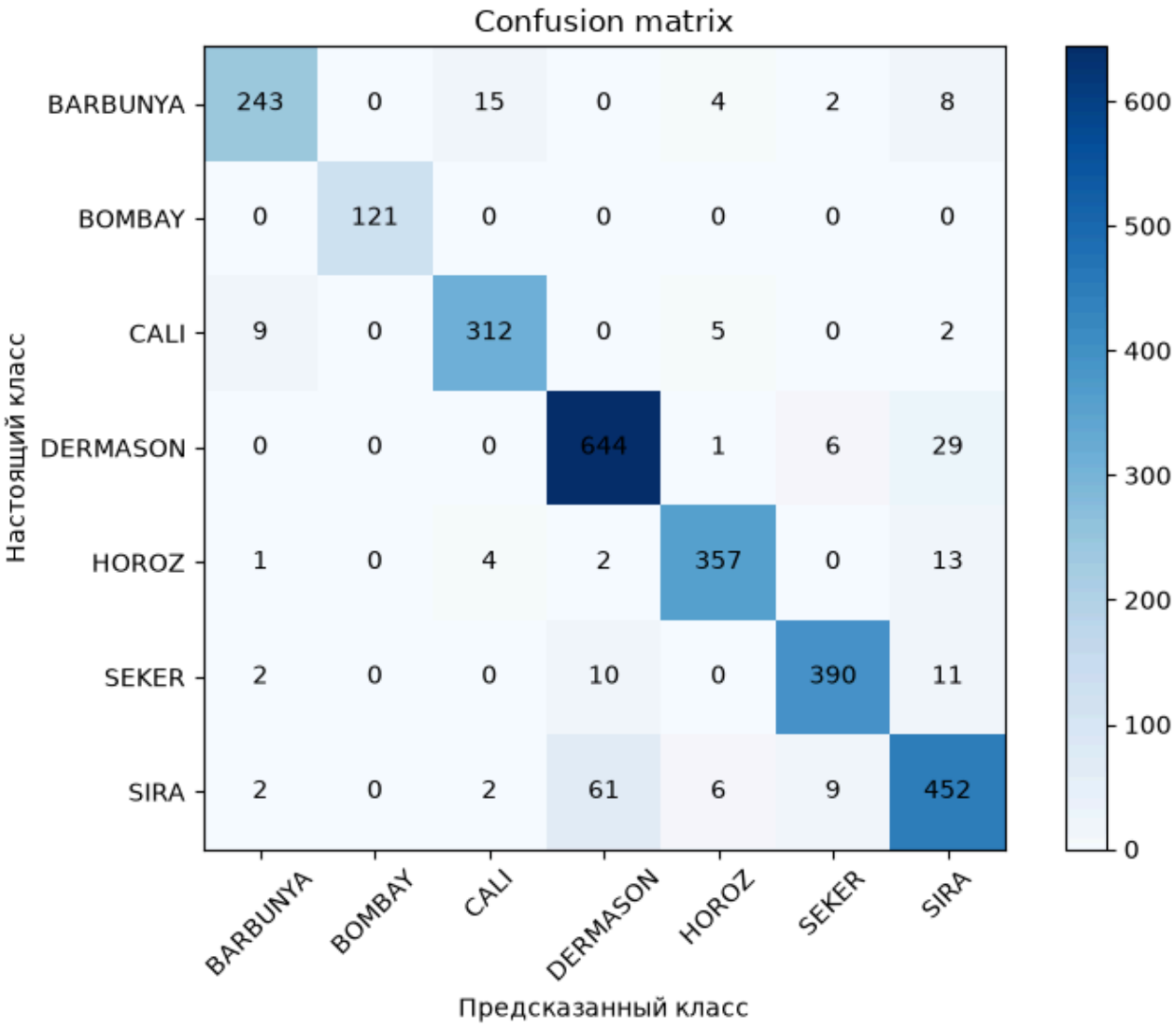


Рис. 4. Матрица ошибок KNN на test

В confusion matrix строки соответствуют настоящим классам, а столбцы - предсказанным.

Значения на главной диагонали являются правильными ответами. Значения вне диагонали показывают ошибки между классами.

Например, класс `SIRA` чаще всего путался с `DERMASON`. Класс `BOMBAY` был распознан без ошибок: 121 объект из 121.

5.5. Вывод по части B

Лучший результат показал KNN с $k = 8$, Евклидовым расстоянием и масштабированными признаками.

Scaling значительно улучшил качество, так как без него признаки с большими числовыми значениями сильнее влияли на расстояние.

6. C. KNN для регрессии

6.1. Датасет

Для регрессии использовался `SeoulBikeData.csv`.

Целевая переменная:

```
y = df["Rented Bike Count"]
```

Дата была разделена на отдельные числовые признаки. Категориальные признаки были преобразованы через `get_dummies`.

Как и в части B, были подготовлены данные:

- без scaling;
- со scaling.

Все параметры preprocessing вычислялись только по `train`.

6.2. Принцип работы

При регрессии KNN выбирает k ближайших объектов и возвращает среднее их целевых значений:

$$\hat{y} = \frac{1}{k} \sum_{i \in N_{k(x)}} y_i$$

Здесь $N_{k(x)}$ - множество k ближайших соседей объекта x.

6.3. Подбор параметров

На validation сравнивались разные значения k, две метрики расстояния и варианты с масштабированием и без него.

Лучший вариант:

- $k = 15$;
- Manhattan distance;
- данные со scaling.

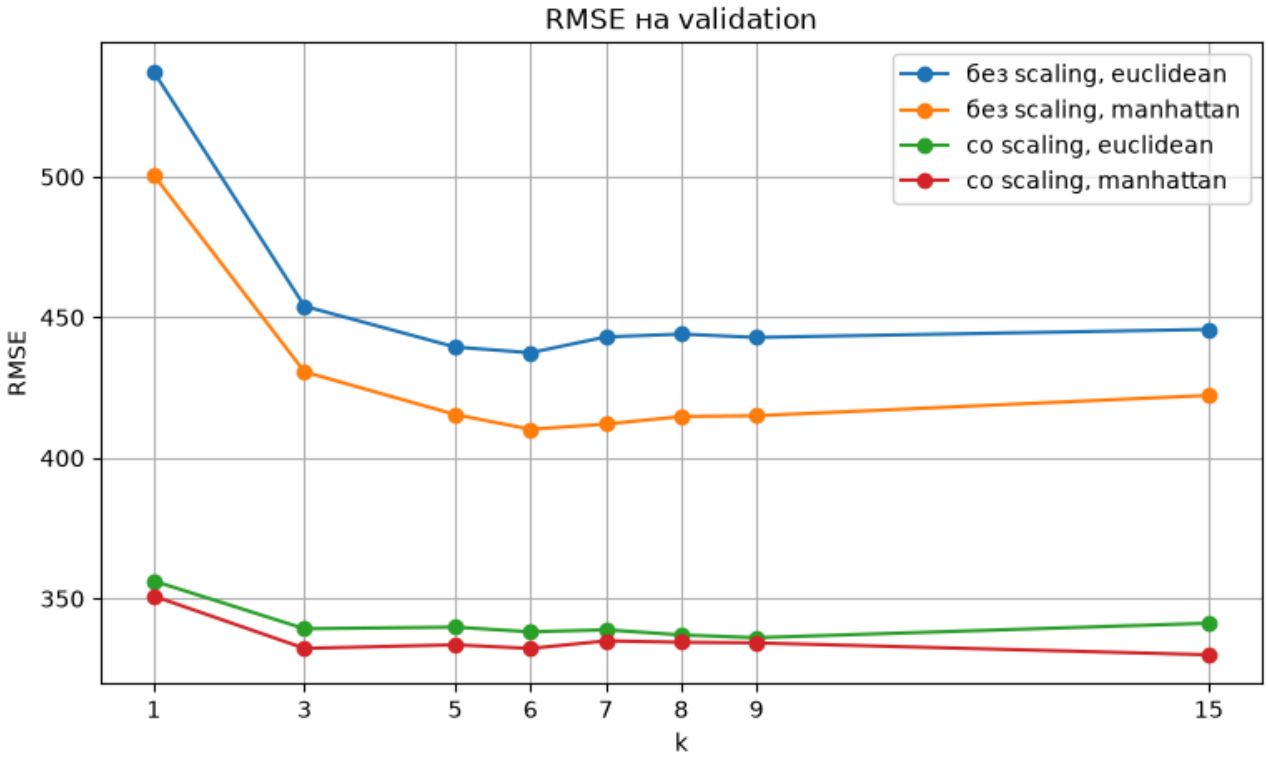


Рис. 5. Сравнение вариантов KNN-регрессии на validation

6.4. Итоговые метрики

Для оценки использовались MAE, RMSE и R^2.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2}$$

На test модель показала:

$$\text{MAE} = 216.3$$

$$\text{RMSE} = 319.5$$

$$R^2 = 0.767$$

В среднем модель ошибается примерно на 216 велосипедов. Более высокое значение RMSE показывает, что присутствуют отдельные крупные ошибки.

Значение $R^2 = 0.767$ означает, что модель объясняет около 76.7% разброса целевой переменной.

6.5. Сравнение с baseline

Baseline всегда предсказывает среднее значение `y_train`:

```
baseline_pred = np.full(len(y_test), y_train.mean())
```

Результаты baseline:

- MAE = 533.1;
- RMSE = 662.4;
- $R^2 \approx 0$.

KNN заметно превосходит baseline по всем метрикам. Следовательно, модель действительно находит зависимость между признаками и количеством аренд.

TODO: Вставить таблицу сравнения KNN и baseline.

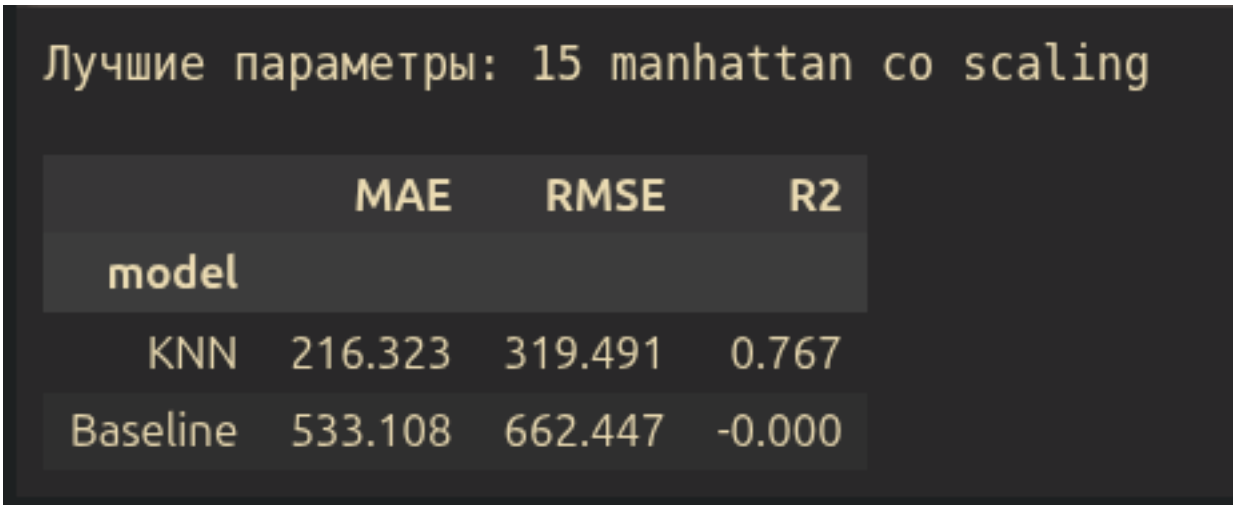


Рис. 6. Сравнение KNN-регрессии с baseline

6.6. Вывод по части C

Лучший результат показал KNN с $k = 15$, Манхэттенским расстоянием и масштабированными признаками.

Модель значительно лучше простого baseline и объясняет большую часть разброса количества аренд.

7. Общий вывод

В лабораторной работе были исследованы регуляризация и метод ближайших соседей.

Собственные реализации моделей вынесены в отдельные файлы. Разделение данных, preprocessing, подбор гиперпараметров и вычисление метрик выполнены вручную с использованием `NumPy` и `Pandas`.

Основные результаты:

- L2-регуляризация немного улучшила линейную регрессию;
- для логистической регрессии разные виды регуляризации дали близкие результаты;
- scaling оказался важен для KNN;
- KNN успешно решил задачи многоклассовой классификации и регрессии;
- обе итоговые KNN-модели заметно превосходят простые baseline-решения.